

Capstone Project

Integrity Risk Assessment Agent

Reasoning, Memory, Tooling and Improvements

The integrity Risk Assessment Agent is designed to answer complex, high-stakes questions such as *“Is Company X safe to partner with?”* by combining dynamic reasoning, multi-step tool orchestration, and adaptive memory. Unlike a prompt-only language model, this agent operates as a reasoning-driven system that gathers real evidence, analyzes it, and synthesizes a defensible conclusion. The following sections describe the core design elements that enable this behavior.

Reasoning Loop, How the Agent Thinks and Acts

The agent uses a dynamic reasoning loop inspired by ReAct, structured around the cycle **Think → Act → Observe → Revise → Conclude**. This loop ensures that the agent does not jump directly to an answer but instead follows a disciplined, evidence-driven process.

Interpret (Think)

The agent begins by interpreting the user’s intent and decomposing the question into relevant risk dimensions—financial stability, regulatory exposure, sanctions, ESG controversies, reputational signals, and operational resilience. This step defines what evidence is required and in what priority.

Plan (Think)

The agent generates a short, adaptive plan that identifies which data sources to query, which models to run, and what order of operations makes sense. The plan is not static; it is a hypothesis that will evolve as new information arrives.

Act

The agent executes the plan by calling external tools: sanctions APIs, news retrieval systems, regulatory databases, market data feeds, and forecasting models. Each action produces structured observations with provenance.

Observe

The agent analyzes the returned data—extracting entities, events, sentiment, dates, and confidence scores. Observations are normalized into structured evidence items.

Revise (Think)

Based on observations, the agent updates its plan. If sanctions data reveals a watchlist flag, the agent may pivot to enforcement actions. If news sentiment is sharply negative, it

may expand the time window or run deeper ESG analysis. If data is missing or contradictory, the agent triggers fallback strategies or escalates uncertainty.

Handling conflicting signals during the reasoning loop

When the agent detects contradictory information (e.g., sanctions list vs. news report), it triggers a conflict-resolution routine. The agent ranks sources by authority (official lists > regulators > reputable news > secondary sources), checks timestamps for update delays, and performs entity resolution to confirm both signals refer to the same entity.

If needed, it retrieves additional evidence such as regulator press releases or expanded news windows. The agent then weighs authority, recency, and corroboration to form a conclusion with an explicit confidence level. If uncertainty remains, it escalates the case and recommends manual review.

Conclude

Once sufficient evidence is gathered, the agent synthesizes a final assessment with justification, confidence levels, and recommended next steps. The conclusion is grounded in verifiable data, not pattern-based inference.

This dynamic loop allows the agent to adapt its reasoning in real time, mirroring how a human analyst investigates risk.

Memory Requirements, Short-term and Long-term

The agent requires both short-term and long-term memory, each serving a distinct purpose.

Short-Term Memory (Working Memory)

Short-term memory stores all context relevant to the current assessment:

- the user's question
- the active reasoning plan
- retrieved documents and structured data
- intermediate model outputs
- unresolved conflicts or missing evidence

This memory enables multi-step reasoning. Without it, the agent would repeat steps, lose track of evidence, or fail to adapt dynamically. Short-term memory is cleared after the assessment ends.

Long-Term Memory

Long term memory allows the agent to learn across assessments and maintain continuity over time. It stores only meaningful, stable, reusable facts rather than raw API responses. Examples include:

- historical risk assessments for each company
- past controversies, sanctions, and enforcement actions
- user preferences (risk tolerance, explanation style)
- model calibration data (false positives/negatives, threshold adjustments)
- watchlists and monitoring rules

These items are first extracted and summarized by the agent, then encoded as embeddings and stored in a vector database. This semantic memory enables the agent to retrieve relevant past facts based on meaning, not just exact matches. Long term memory is consulted at the start of an assessment (to provide historical context) and again during conflict resolution (to see whether similar issues occurred before), but it never replaces real time evidence from external tools. New information is written to long term memory after an assessment is completed, once conflicts are resolved and facts are deemed stable.

This memory supports trend analysis, adaptive thresholding, and personalized behavior. It transforms the agent from a one-off evaluator into a continuously improving risk intelligence system.

Using historical context in conflicts

Long-term memory helps the agent interpret conflicting signals by comparing them with historical patterns, past sanctions status, prior controversies, or earlier assessments. This context allows the agent to determine whether a conflict represents a new development or part of an ongoing trend, improving confidence and reducing overreaction to isolated or unverified signals.

Tool Usage, Why Tools Are Essential

The agent relies on external tools because many tasks cannot be performed by an LLM alone.

Example Task: Sanctions and Enforcement Check

To determine whether a company is sanctioned or under investigation, the agent must:

1. Query real-time sanctions lists
2. Perform entity resolution to match name variants
3. Retrieve enforcement actions from regulatory databases
4. Analyze recent news for sanction-related events

These steps require:

- **real-time retrieval** (LLMs cannot access live data)
- **structured computation** (LLMs cannot run numerical models)
- **entity matching** (LLMs cannot reliably disambiguate similar names)
- **grounding** (LLMs hallucinate when asked factual questions)

Tools provide the grounding, computation, and reliability that a language model alone cannot achieve.

Example Task: Financial Distress Forecasting

The agent may ingest market data and run a forecasting model to estimate liquidity risk. This requires numerical computation and time-series analysis, capabilities outside the scope of an LLM.

Tool use during conflicts

When signals conflict, the agent invokes additional tools to gather clarifying evidence. This may include entity-resolution services, cross-jurisdiction sanctions checks, regulator press releases, or expanded news retrieval. These tool-driven actions ensure the agent resolves conflicts using fresh, authoritative data rather than relying on assumptions or static text generation.

In short, tools give the agent the ability to act in the real world, not just generate text.

Improvements Over a Prompt-Only Approach

A prompt-only LLM is insufficient for integrity risk assessment due to several failure modes:

Failure Mode: Hallucinated or Outdated Facts

A prompt-only LLM might confidently state that a company is not sanctioned, even if it was added to a sanctions list last week. This is unacceptable in compliance contexts.

The agent solves this by querying real-time sanctions APIs and verifying evidence with timestamps and provenance.

Failure Mode: No Multi-Step Reasoning

A simple prompt cannot orchestrate a workflow involving multiple data sources, fallback strategies, and conflict resolution.

The agent solves this through its dynamic reasoning loop, which adapts based on observations.

Failure Mode: No Memory

A prompt-only LLM cannot track historical risk trends or learn from past outcomes.

The agent solves this with long-term memory that stores historical assessments, user preferences, and model calibration data.

Failure Mode: No Computation

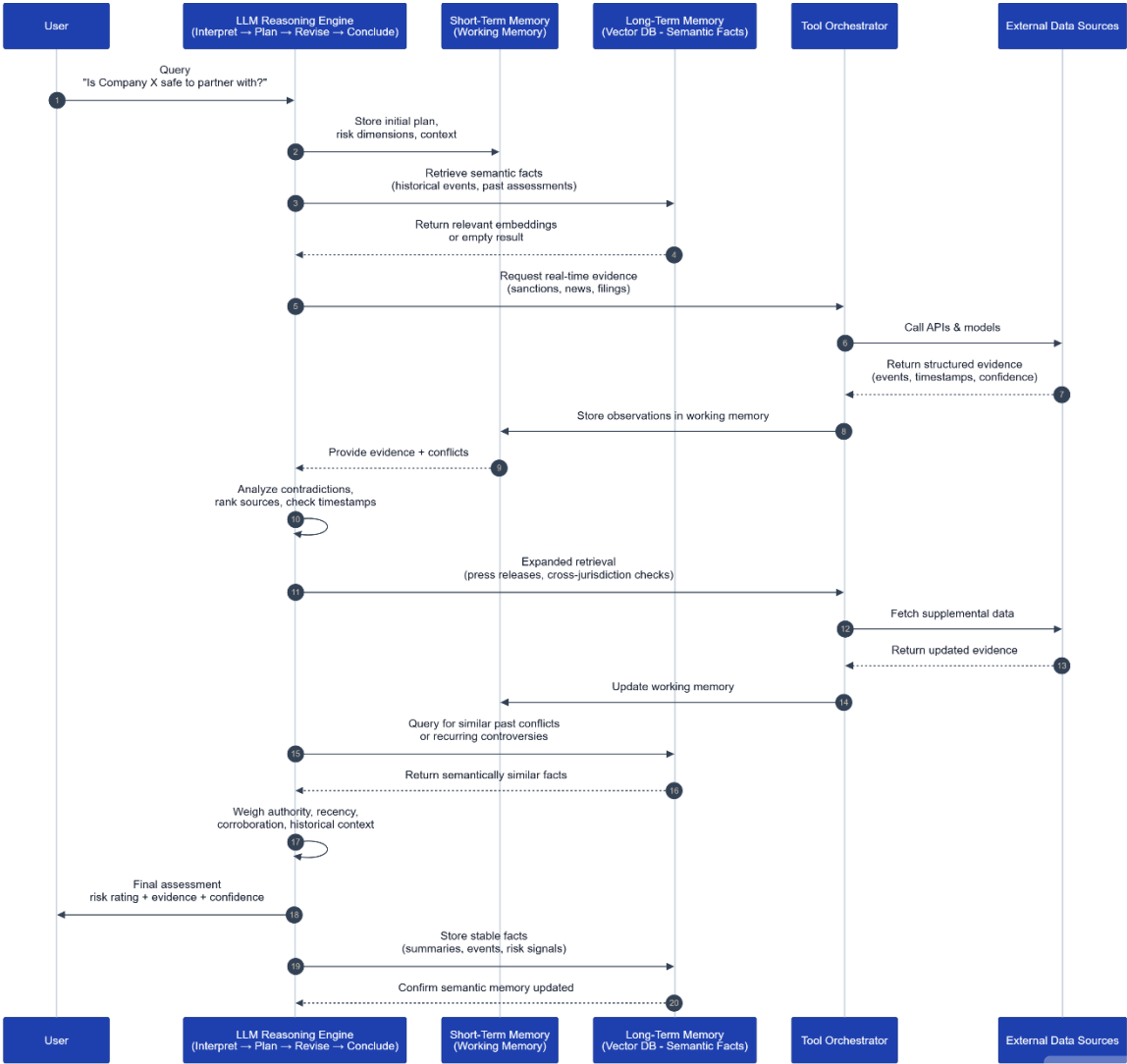
LLMs cannot run forecasting models, anomaly detection, or structured scoring frameworks.

Failure Mode: Conflicting Signals

Prompt-only LLMs often mishandle conflicting information, choosing one source arbitrarily or hallucinating a synthesis. The agent avoids this by applying a structured conflict-resolution protocol: ranking sources, checking timestamps, verifying identity, gathering more evidence, and assigning confidence levels. This produces transparent, defensible conclusions that a prompt-only model cannot reliably generate.

The agent solves this by delegating computation to external tools.

Sequence Diagram detailing high-level execution flow,



MermaidEditor.io

Note, I used Microsoft Copilot as a writing support tool to help structure and refine the language in this project description. All ideas, analysis, and final content decisions are my own.